



- Tips for getting started:
1. Ask questions, edit files, or run commands.
  2. Be specific for the best results.
  3. Create **GEMINI.md** files to customize your interactions with Gemini.
  4. **/help** for more information.

iGemini CLI update available! 0.11.3 → 0.12.0  
Installed via Homebrew. Please update with "brew upgrade".

> que es testing de caja negra y caja blanca

- + El testing de caja negra (black box testing) es un método de prueba de software que examina la funcionalidad de una aplicación sin mirar su estructura interna o implementación. Se enfoca en las entradas y salidas del sistema, verificando que el software cumpla con los requisitos especificados.

El testing de caja blanca (white box testing), también conocido como testing de caja de cristal o testing estructural, es un método de prueba de software que examina la estructura interna, el diseño y la implementación del sistema. Los probadores tienen conocimiento del código fuente, la arquitectura y el diseño del software, y utilizan este conocimiento para diseñar casos de prueba que cubran rutas de código, condiciones y declaraciones.

> Estudio Ingenieria en Sistemas de Informacion. Necesito aprender todo sobre testing caja negra y caja blanca. Explica detalladamente y da ejemplos especificos de su implementacion

- + Excelente. Dado que estudias Ingeniería en Sistemas de Información, profundizaremos en los aspectos técnicos, metodologías y ejemplos prácticos de ambos tipos de testing.

### Testing de Caja Negra (Black-Box Testing)

Este enfoque se centra en el **qué** hace el sistema, no en el **cómo** lo hace. El tester no necesita conocer el lenguaje de programación, la arquitectura interna ni la estructura del código. Se interactúa con la aplicación como lo haría un usuario final.

**Objetivo Principal:** Verificar que el software cumple con los requisitos funcionales y no funcionales especificados.

*Técnicas Fundamentales de Caja Negra y sus Ejemplos:*

1. **Partición de Equivalencia:**
  - \* **Concepto:** Se divide el conjunto de todas las posibles entradas en "clases de equivalencia" o grupos que se espera que el sistema procese de la misma manera. Se prueba un solo valor de cada grupo, asumiendo que si funciona para uno, funcionará para todos los demás en esa partición.
  - \* **Ejemplo:** Un campo de texto que acepta edades entre 18 y 60 años.
    - \* **Clase de equivalencia válida:** Cualquier número entre 18 y 60 (ej. **35**).
    - \* **Clase de equivalencia inválida (menor):** Cualquier número menor a 18 (ej. **17**).
    - \* **Clase de equivalencia inválida (mayor):** Cualquier número mayor a 60 (ej. **61**).
    - \* **Clase de equivalencia inválida (no numérico):** Cualquier texto (ej. **"abc"**).
  - \* **Implementación:** Se diseñan casos de prueba para cada una de estas clases: **35** (debería ser aceptado), **17** (debería ser rechazado), **61** (debería ser rechazado), **"abc"** (debería mostrar un error).
2. **Análisis de Valores Límite (Boundary Value Analysis):**
  - \* **Concepto:** Es una extensión de la partición de equivalencia. Se enfoca en probar los "límites" o "bordes" de cada partición, ya que es en estos puntos donde suelen ocurrir los errores.
  - \* **Ejemplo:** Usando el mismo campo de edad (18-60).
    - \* **Límites de la clase válida:** **18** y **60**.
    - \* **Valores inmediatamente adyacentes a los límites:** **17** (justo por debajo del mínimo) y **61** (justo por encima del máximo).
  - \* **Implementación:** Se crean casos de prueba específicos para los valores: **17**, **18**, **60**, **61**. Esto es más preciso que solo probar con **35**.
3. **Tabla de Decisión:**
  - \* **Concepto:** Se utiliza cuando la salida del sistema depende de una combinación de múltiples condiciones de entrada. Se crea una tabla que mapea todas las combinaciones de condiciones (verdadero/falso) a sus acciones o resultados esperados.
  - \* **Ejemplo:** Un formulario de login con las condiciones "Usuario Correcto" y "Contraseña Correcta".

| Caso | Condición: Usuario Correcto | Condición: Contraseña Correcta | Acción/Resultado Esperado                   |
|------|-----------------------------|--------------------------------|---------------------------------------------|
| 1    | Falso                       | Falso                          | Mensaje: "Usuario o contraseña incorrectos" |
| 2    | Falso                       | Verdadero                      | Mensaje: "Usuario o contraseña incorrectos" |
| 3    | Verdadero                   | Falso                          | Mensaje: "Usuario o contraseña incorrectos" |
| 4    | Verdadero                   | Verdadero                      | Acceso concedido                            |

- \* **Implementación:** Se crea un caso de prueba para cada una de las 4 combinaciones para asegurar que el sistema se comporta como se espera en todos los escenarios.

4. **Pruebas de Transición de Estado:**
  - \* **Concepto:** Ideal para sistemas que tienen diferentes "estados" y transiciones entre ellos. Se modela el sistema como una máquina de estados y se prueban las transiciones de un estado a otro.
  - \* **Ejemplo:** Un cajero automático (ATM).
    - \* **Estados:** **Inactivo**, **Esperando PIN**, **Acceso a cuenta**, **Transacción finalizada**.
    - \* **Transiciones:**
      - \* **Inactivo** -> **Esperando PIN** (al insertar tarjeta).
      - \* **Esperando PIN** -> **Acceso a cuenta** (al ingresar PIN correcto).
      - \* **Esperando PIN** -> **Inactivo** (al ingresar PIN incorrecto 3 veces).
      - \* **Acceso a cuenta** -> **Transacción finalizada** (al completar una operación).
  - \* **Implementación:** Se diseñan pruebas que fuercen cada una de estas transiciones para verificar que el sistema cambia de estado correctamente.

---

### Testing de Caja Blanca (White-Box Testing)

Este enfoque requiere un conocimiento profundo del funcionamiento interno del sistema. El tester necesita leer el código fuente para diseñar los casos de prueba. Se centra en el **cómo** funciona el sistema.

**Objetivo Principal:** Asegurar que todas las rutas, ramas y condiciones del código se ejecutan y funcionan como se espera. Garantizar la calidad y la seguridad del código.

Para los ejemplos, usemos la siguiente función en pseudocódigo que calcula un descuento:

```
1 FUNCTION calcularDescuento(esMiembro, montoCompra)
2   1. descuento = 0
3   2. IF esMiembro == TRUE AND montoCompra > 100 THEN
4     3. descuento = 0.10
5   4. ELSE IF montoCompra > 50 THEN
6     5. descuento = 0.05
7   6. END IF
8   7. RETURN descuento
9 END FUNCTION
```

1. Cobertura de Sentencias (Statement Coverage):

- \* **Concepto:** El objetivo es ejecutar cada línea de código ejecutable al menos una vez.
- \* **Ejemplo:** Para la función `calcularDescuento`:
  - \* **Caso de prueba 1:** `calcularDescuento(TRUE, 150)`.  
\* Este caso ejecuta las líneas 1, 2, 3, y 7.
  - \* **Caso de prueba 2:** `calcularDescuento(FALSE, 75)`.  
\* Este caso ejecuta las líneas 1, 2, 4, 5, y 7.
- \* **Implementación:** Con estos dos casos, hemos ejecutado todas las líneas del 1 al 7. Logramos un 100% de cobertura de sentencias.

2. Cobertura de Decisión o Rama (Decision/Branch Coverage):

- \* **Concepto:** Más estricto que el anterior. El objetivo es que cada posible resultado de una decisión (cada rama `IF`, `ELSE`, `CASE`) se ejecute al menos una vez. Es decir, cada condición debe evaluarse tanto a `TRUE` como a `FALSE`.
- \* **Ejemplo:**
  - \* **Decisión 1 (línea 2):** `esMiembro == TRUE AND montoCompra > 100`
  - \* **Decisión 2 (línea 4):** `montoCompra > 50`
  - \* **Caso de prueba 1:** `calcularDescuento(TRUE, 150)`.  
\* Decisión 1 es `TRUE`. Ejecuta la rama del `IF` principal.
  - \* **Caso de prueba 2:** `calcularDescuento(FALSE, 75)`.  
\* Decisión 1 es `FALSE`. Decisión 2 es `TRUE`. Ejecuta la rama del `ELSE IF`.
  - \* **Caso de prueba 3:** `calcularDescuento(FALSE, 20)`.  
\* Decisión 1 es `FALSE`. Decisión 2 es `FALSE`. No ejecuta ninguna rama de descuento.
- \* **Implementación:** Con estos 3 casos, hemos cubierto todos los resultados posibles de las decisiones.

3. Cobertura de Condición (Condition Coverage):

- \* **Concepto:** Se enfoca en las sub-condiciones dentro de una decisión compleja. Cada sub-condición debe ser evaluada como `TRUE` y `FALSE` al menos una vez.
- \* **Ejemplo:** En la Decisión 1 (`esMiembro == TRUE AND montoCompra > 100`), tenemos dos sub-condiciones:
  1. `esMiembro == TRUE`
  2. `montoCompra > 100`
- \* **Implementación:**
  - \* **Caso de prueba 1:** `calcularDescuento(TRUE, 150)`.  
\* Sub-condición 1 es `TRUE`. Sub-condición 2 es `TRUE`.
  - \* **Caso de prueba 2:** `calcularDescuento(FALSE, 40)`.  
\* Sub-condición 1 es `FALSE`. Sub-condición 2 es `FALSE`.
- \* Con estos dos casos, ambas sub-condiciones han sido `TRUE` y `FALSE`, logrando 100% de cobertura de condición. *Nota: Esto no garantiza la cobertura de decisión.* Por ejemplo, el resultado `FALSE` de la Decisión 1 no se ha probado con `montoCompra > 100`.

4. Cobertura de Ruta (Path Coverage):

- \* **Concepto:** La técnica más exhaustiva. El objetivo es probar cada una de las posibles rutas de ejecución a través del código.
- \* **Ejemplo:** En nuestra función, las rutas posibles son:
  1. Líneas 1, 2, 3, 7 (IF principal es `TRUE`).
  2. Líneas 1, 2, 4, 5, 7 (IF principal es `FALSE`, ELSE IF es `TRUE`).
  3. Líneas 1, 2, 4, 6, 7 (IF principal es `FALSE`, ELSE IF es `FALSE`).
- \* **Implementación:** Se necesitaría un caso de prueba para cada ruta:
  - \* **Ruta 1:** `calcularDescuento(TRUE, 150)`
  - \* **Ruta 2:** `calcularDescuento(FALSE, 75)`
  - \* **Ruta 3:** `calcularDescuento(FALSE, 40)`

Resumen y Cuándo Usar Cada Uno

| Característica          | Testing de Caja Negra                           | Testing de Caja Blanca                                  |
|-------------------------|-------------------------------------------------|---------------------------------------------------------|
| **Conocimiento del c... | No requerido                                    | Requerido                                               |
| Realizado por           | Testers, QA, usuarios finales                   | Desarrolladores, Testers técnicos                       |
| Nivel de prueba         | Generalmente en niveles altos (Sistema, Acep... | Generalmente en niveles bajos (Unitario, Integración)   |
| Objetivo                | Validar comportamiento funcional (requisitos)   | Validar estructura interna y calidad del código         |
| Ventaja principal       | Imparcialidad, simula al usuario real           | Exhaustividad, encuentra errores ocultos en la lógica   |
| Desventaja principal    | No puede encontrar errores en rutas no utili... | No puede encontrar requisitos faltantes o mal interp... |

**Conclusión para tu carrera:** No se trata de elegir uno sobre el otro. Un buen proceso de testing combina ambos enfoques.

- \* Los desarrolladores aplican **testing de caja blanca** (especialmente pruebas unitarias) para asegurar que el código que escriben es robusto.
- \* El equipo de QA aplica **testing de caja negra** para asegurar que el software, como un todo, cumple con lo que el cliente y los usuarios esperan, sin importar cómo se haya construido.

> /quit

Agent powering down. Goodbye!

Interaction Summary

Session ID:9dc54341-1347-4545-9118-15e750031f1f

Tool Calls:0 ( ✓ 0 x 0 )

Success Rate:0.0%

Performance

Wall Time:20m 9s

Agent Active:40.0s

» API Time:40.0s (100.0%)